

Here is the **complete, fully functional, and commented code** for your Python Full-Stack Task Management Web Application.

To make it incredibly professional and ensure you score perfectly on your CodTech internship submission, this project uses **Flask (Python)** for the backend, an integrated **SQLite database**, and a clean, responsive single-file user interface styling via **Tailwind CSS** (loaded via CDN, so you don't have to manage complex frontend build folders).

Follow the layout below to build your files.

Project Directory Setup

Create these exact folders and files on your computer:

```
task-manager-app/
|
├── app.py                # Main Python Flask backend (with comments)
├── requirements.txt      # Project dependencies
├── README.md             # Documentation (as required by Image 2)
|
├── templates/           # Folder for HTML templates
│   └── index.html        # Full-stack Frontend interface
|
└── screenshots/         # Store your app screenshots here
    └── output.png
```

Complete & Commented Source Code

1. requirements.txt

```
Flask==3.0.3
Flask-SQLAlchemy==3.1.1
```

2. app.py (Backend Logic & Database Configuration)

```
import os
from flask import Flask, render_template, request, redirect, url_for, flash
from flask_sqlalchemy import SQLAlchemy

# Initialize the Flask Application
app = Flask(__name__)

# Secret key required for securely handling user session flash messages
app.config['SECRET_KEY'] = 'codtech_secret_key_12345'

# Define the SQLite database path (creates 'tasks.db' inside the
```

```

project directory)
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///tasks.db'
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False

# Initialize the SQLAlchemy Object-Relational Mapper (ORM)
db = SQLAlchemy(app)

# -----
# DATABASE MODEL (Defines the 'Task' table structure)
# -----
class Task(db.Model):
    id = db.Column(db.Integer, primary_key=True)          #
    Unique ID for each task
    title = db.Column(db.String(100), nullable=False)      #
    Title of the task (Required)
    description = db.Column(db.String(300), nullable=True) #
    Short explanation (Optional)
    status = db.Column(db.String(20), default='Pending')  #
    Status: Pending or Completed

    def __repr__(self):
        return f'<Task {self.id}: {self.title}>'

# Automatically create the database file and tables before handling
the first request
with app.app_context():
    db.create_all()

# -----
# APP ROUTES & CRUD OPERATIONS
# -----

# 1. READ: Display all tasks on the dashboard
@app.route('/')
def index():
    # Fetch all tasks from the SQLite database
    all_tasks = Task.query.order_by(Task.id.desc()).all()
    # Render the frontend page and pass the list of tasks to it
    return render_template('index.html', tasks=all_tasks)

# 2. CREATE: Add a new task to the database
@app.route('/add', methods=['POST'])
def add_task():
    # Capture form data submitted from the UI
    title = request.form.get('title')
    description = request.form.get('description')

```

```

# Simple data validation rule
if not title:
    flash('Task Title is required!', 'error')
    return redirect(url_for('index'))

# Instantiate a new Task database row
new_task = Task(title=title, description=description)

try:
    db.session.add(new_task) # Stage the object
    db.session.commit()      # Write permanently to SQLite
    flash('Task added successfully!', 'success')
except Exception as e:
    db.session.rollback()    # Roll back changes if an error
occurs
    flash(f'Error saving task: {str(e)}', 'error')

return redirect(url_for('index'))

# 3. UPDATE: Toggle the completion status of a task
@app.route('/update/<int:task_id>')
def update_task(task_id):
    # Fetch the task by ID, or throw a 404 Error if not found
    task = Task.query.get_or_404(task_id)

    # Toggle status values between Pending and Completed
    if task.status == 'Pending':
        task.status = 'Completed'
    else:
        task.status = 'Pending'

    db.session.commit() # Save changes
    flash(f'Task "{task.title}" status updated!', 'success')
    return redirect(url_for('index'))

# 4. DELETE: Remove a task from the board
@app.route('/delete/<int:task_id>')
def delete_task(task_id):
    task = Task.query.get_or_404(task_id)

    try:
        db.session.delete(task) # Delete row
        db.session.commit()     # Commit operation
        flash('Task deleted successfully.', 'success')
    except Exception as e:

```

```

        db.session.rollback()
        flash('Could not delete the task.', 'error')

    return redirect(url_for('index'))

# Run the application locally on Port 5000 with interactive debug logs
turned on
if __name__ == '__main__':
    app.run(debug=True)

```

3. templates/index.html (Frontend User Interface)

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,
initial-scale=1.0">
    <title>Full-Stack Python Task Manager</title>
    <script src="https://cdn.tailwindcss.com"></script>
</head>
<body class="bg-slate-50 text-slate-800 font-sans min-h-screen">

    <div class="max-w-4xl mx-auto py-12 px-4">
        <header class="text-center mb-10">
            <h1 class="text-4xl font-extrabold text-indigo-600
tracking-tight">Full-Stack Task Dashboard</h1>
            <p class="text-sm text-slate-500 mt-2">Built with Python,
Flask, and SQLite for CodTech Project Submission</p>
        </header>

        {% with messages = get_flashed_messages(with_categories=true)
%}
            {% if messages %}
                {% for category, message in messages %}
                    <div class="mb-6 p-4 rounded-lg text-sm
font-medium {% if category == 'error' %} bg-red-100 text-red-700 {%
else %} bg-emerald-100 text-emerald-800 {% endif %}">
                        {{ message }}
                    </div>
                {% endfor %}
            {% endif %}
        {% endwith %}

        <div class="grid grid-cols-1 md:grid-cols-3 gap-8">
            <div class="bg-white p-6 rounded-xl shadow-sm border

```

```

border-slate-200 h-fit">
    <h2 class="text-lg font-bold text-slate-900
mb-4">Create New Task</h2>
    <form action="/add" method="POST" class="space-y-4">
        <div>
            <label class="block text-xs font-bold
uppercase tracking-wider text-slate-500 mb-1">Task Title *</label>
            <input type="text" name="title"
placeholder="e.g., Fix Auth Middleware" class="w-full px-3 py-2 border
border-slate-300 rounded-lg focus:outline-none focus:ring-2
focus:ring-indigo-500 text-sm" required>
        </div>
        <div>
            <label class="block text-xs font-bold
uppercase tracking-wider text-slate-500 mb-1">Description</label>
            <textarea name="description" rows="3"
placeholder="Brief notes about this work item..." class="w-full px-3
py-2 border border-slate-300 rounded-lg focus:outline-none
focus:ring-2 focus:ring-indigo-500 text-sm"></textarea>
        </div>
        <button type="submit" class="w-full bg-indigo-600
hover:bg-indigo-700 text-white font-semibold py-2 px-4 rounded-lg
transition text-sm">
            + Add Task To Board
        </button>
    </form>
</div>

<div class="md:col-span-2 space-y-4">
    <h2 class="text-xl font-bold text-slate-900 flex
items-center justify-between">
        <span>Task Backlog</span>
        <span class="text-xs font-semibold bg-indigo-100
text-indigo-700 px-2.5 py-1 rounded-full">{{ tasks|length }}
Items</span>
    </h2>

    {% if tasks|length == 0 %}
        <div class="bg-white text-center py-12 px-4
rounded-xl border border-dashed border-slate-300 text-slate-400
text-sm">
            No tasks found. Use the sidebar module to
            create your first tracking entry!
        </div>
    {% else %}
        {% for task in tasks %}
            <div class="bg-white p-5 rounded-xl shadow-sm
border border-slate-200 flex items-start justify-between gap-4 {% if

```

```

task.status == 'Completed' %} opacity-60 bg-slate-50 {% endif %}">
    <div class="space-y-1">
        <div class="flex items-center gap-2">
            {% if task.status == 'Completed'
%}

                <span class="inline-block
w-2.5 h-2.5 rounded-full bg-emerald-500"></span>
                <h3 class="text-base font-bold
text-slate-900 line-through">{{ task.title }}</h3>
                {% else %}
                    <span class="inline-block
w-2.5 h-2.5 rounded-full bg-amber-500"></span>
                    <h3 class="text-base font-bold
text-slate-900">{{ task.title }}</h3>
                {% endif %}
            </div>
            {% if task.description %}
                <p class="text-sm text-slate-600
pl-4.5">{{ task.description }}</p>
                {% endif %}
                <div class="pt-1.5 pl-4.5">
                    <span class="text-xs px-2 py-0.5
rounded-md font-medium {% if task.status == 'Completed' %}
bg-emerald-50 text-emerald-700 {% else %} bg-amber-50 text-amber-700
{% endif %}">
                        {{ task.status }}
                    </span>
                </div>
            </div>

            <div class="flex items-center gap-2">
                <a href="/update/{{ task.id }}"
class="text-xs font-semibold px-3 py-1.5 rounded-md border
border-slate-300 hover:bg-slate-50 transition">
                    {% if task.status == 'Completed'
%} Reopen {% else %} Complete {% endif %}
                </a>
                <a href="/delete/{{ task.id }}"
class="text-xs font-semibold px-3 py-1.5 rounded-md bg-red-50
text-red-600 hover:bg-red-100 transition" onclick="return
confirm('Permanently drop this task record?')">
                    Delete
                </a>
            </div>
        </div>
    {% endfor %}
{% endif %}
</div>

```

```
        </div>
    </div>

</body>
</html>
```



Running the Project Locally

1. Install Python packages using terminal commands:
`pip install -r requirements.txt`
2. Fire up the backend:
`python app.py`
3. Open your web browser and head to: **`http://127.0.0.1:5000/`**
4. Perform testing actions (Add a task, Complete it, Delete one) to populate database changes.
5. Take a clear window screenshot of your dashboard app UI and save it inside the `/screenshots` folder before publishing online to GitHub!